GUJARAT TECHNOLOGICAL UNIVERSITY
Chandkheda, Ahmedabad
Affiliated

Sardar Vallabhbhai Patel Institute of Technology,
Vasad

A Project On
PBL Activity: Research Writing
IPL Match & Player Performance Analysis

Under the Subject
Python for Data Science (BE05000231)

BE 3rd YEAR, SEM 5 (Computer Engineering branch)

Submitted by:
Bardanwala Husain (240410107008)
Bardanwala Taha (240410107009)
Bhalani Miral (240410107011)
Nikunj Dhobi (240410107033)

Prof. Keyur Suthar
(Faculty Guide)

Prof. Jayna Shah
(Head of the Department)

TABLE OF CONTENTS

Sr no.	Topic	Page no.
1	Title Page	3
2	Introduction	3
3	Project Definition & Problem Statement / Objectives	6
4	Proposed System / System Design	7
5	Screenshots of Results	11
6	Conclusion and Future Scope	19
7	References	20

1. Title Page

IPL MATCH & PLAYER PERFORMANCE ANALYSIS

A Python-based data analysis of 17 seasons of Indian Premier League matches and ball-by-ball records to find the factors behind team and player performance

Table 1: Project details

Item	Details
Project title	IPL Match & Player Performance Analysis
Activity	PBL – Research Writing
Course outcomes	CO2, CO4
Subject	Python for Data Science (BE05000231)
Dataset	matches.csv (1,095 records, 20 columns) and deliveries.csv (260,920 records, 17 columns), IPL 2008–2024
Technology stack	Python 3, pandas, NumPy, Matplotlib, Seaborn (Jupyter Notebook)
Submitted by	Bardanwala Husain (240410107008), Bardanwala Taha (240410107009), Bhalani Miral (240410107011), Nikunj Dhobi (240410107033)
Class	BE 3rd Year, Semester 5, Computer Engineering
Faculty guide	Prof. Keyur Suthar
Institute	Sardar Vallabhbhai Patel Institute of Technology, Vasad

2. Introduction

Data science is the process of collecting, cleaning, analysing and visualising data to find useful patterns and support decisions. Python is the most widely used language for data science because libraries such as **pandas**, **NumPy**, **Matplotlib** and **Seaborn** make it simple to load a dataset, correct its errors, calculate statistics and draw charts.

The **Indian Premier League (IPL)** is a professional T20 cricket league played every year since 2008. Every match produces a large amount of data: the result, the toss, the venue and, ball by ball, the runs scored, extras conceded and wickets taken. Teams, broadcasters and analysts use this data to judge players, plan strategies and explain results. Cricket data is therefore an ideal subject for practising the complete data-science workflow on a real, large dataset.

Raw cricket data is not clean, however. Over 17 seasons several franchises changed their names, venue names were recorded in different ways, some seasons were labelled with two years (for example **2007/08**), and many columns are empty by design (most balls are not wickets or extras). If these problems are not corrected first, one team is counted as two, averages include super overs and bowlers are credited with run outs.

In this project a Python-based solution is developed in a **Jupyter Notebook**. The two raw files are first **cleaned**, then **merged** into ball-by-ball and innings-level tables, **descriptive statistics** and **outliers** are studied, teams are **segmented**, a set of **charts and a dashboard** is produced, the results are **validated**, and finally the key findings are summarised.

Need of the project: a clean and reliable IPL dataset, fair comparisons between teams and players across 17 seasons, and evidence-based answers to common cricket questions such as how much the toss matters and how scoring has changed.

Python libraries and tools used in this project:

- **Python 3** – the programming language used for the complete solution.
- **pandas** – loading the CSV files into DataFrames, cleaning, merging, groupby and statistics.
- **NumPy** – numerical operations used in the calculations.
- **Matplotlib** – the base plotting library used to set titles and labels and to save charts.
- **Seaborn** – statistical charts such as histograms, box plots, bar plots and heatmaps.
- **Jupyter Notebook / Google Colab** – running the code step by step with the output shown below each cell.

2.1 Cricket Performance Metrics

Raw totals such as runs or wickets depend on how many matches a player or team has played. Rates such as strike rate, economy and win percentage allow a fairer comparison. Table 2 lists the metrics used in this project.

Table 2: Cricket performance metrics used in this project

Metric	Meaning	Formula / use in this project
Innings score	Runs scored by a team in one innings	Sum of total_runs per match and innings
Strike rate (SR)	Scoring speed of a batter	Runs ÷ balls faced × 100 (wides are not balls faced)
Economy	Runs a bowler concedes per over	Runs conceded ÷ legal balls × 6 (byes and leg-byes not charged)
Run rate	Runs scored per over	Runs ÷ legal balls × 6, compared by phase of the innings
Bowler wickets	Dismissals credited to the bowler	Excludes run out, retired hurt, retired out, obstructing the field
Win %	Share of matches a team has won	Wins ÷ matches played × 100
High-score innings	A big total in T20 cricket	Innings score of 180 or more

2.2 Data Cleaning in Data Science

Data cleaning (also called data preprocessing) is the process of finding and correcting errors in a dataset before analysis. Data collected over many years from many sources nearly always contains mistakes and inconsistencies. If they are not fixed, groupings, averages and charts will all be wrong; this idea is often summarised as *garbage in, garbage out*. Table 3 lists the common problems and how they are usually handled.

Table 3: Common data quality problems and handling methods

Problem	Example in IPL data	Common handling method
Missing values	Blank city or winner	Fill from a related column, a constant label or the mode
Structural blanks	No dismissal on a normal ball	Fill with a label such as Not Out / None
Inconsistent categories	Bangalore vs Bengaluru	Map every spelling to one standard label
Renamed entities	Delhi Daredevils → Delhi Capitals	Map old names to the current name

Problem	Example in IPL data	Common handling method
Wrong format	Season 2007/08	Re-derive the value from a reliable column (date)
Outliers	An innings of 2 runs	Detect with IQR; keep if genuine, remove if impossible
Duplicate records	Same match twice	Remove with drop_duplicates()

2.3 Statistical Concepts Used

- **Mean** – the sum of all values divided by the number of values. It is easy to calculate, but extreme values can pull it up or down.
- **Median** – the middle value after sorting. It is not affected by outliers.
- **Mode** – the most frequent value, suitable for categories.
- **Quartiles and IQR** – Q1 is the 25th percentile and Q3 the 75th percentile. IQR = Q3 − Q1 holds the middle 50 % of the data. Values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR are outliers.
- **Pearson correlation (r)** – measures the straight-line relationship between two numeric columns on a scale from −1 to +1. A value near +1 means both rise together, near −1 means one falls as the other rises, and near 0 means no linear relationship.
- **Skewness** – shows whether a distribution has a longer tail on the right (positive skew) or on the left (negative skew).
- **Terciles** – the values that split sorted data into three equal parts; used to segment teams into Low, Medium and High Value.

For missing values the method was chosen column by column (Table 4). Filling the blank city with the **mode** would have placed all 51 matches played in Dubai and Sharjah in Mumbai, so the city was filled from the **venue** instead.

Table 4: Comparison of methods for handling missing values

Method	How it works	Advantage	Disadvantage
Delete rows	Remove every row with a blank	Very simple	Loses whole matches and balls
Mode	Fill with the most frequent value	Works for categories	Wrong city for UAE matches
Related column (used)	Fill city from the venue	Gives the true value	Needs a reliable link
Constant label (used)	No Result, Unknown, Not Out, None, Normal	Keeps the meaning of a structural blank	Must be excluded from counts
Zero (used)	Margin and target of ties / no results	Not-applicable numbers stay numeric	Must not be averaged as a real margin

3. Project Definition & Problem Statement / Objectives

3.1 Project Definition

Table 5: Project definition

Item	Details
PBL activity	Research Writing (CO2, CO4)
Domain	Data science / sports (cricket) analytics
Project title	IPL Match & Player Performance Analysis
Users	Team analysts, coaches, broadcasters, cricket fans
Input data	matches.csv – one row per match; deliveries.csv – one row per ball bowled
Derived features	season (year), innings table, phase, fours, sixes, dot balls, strike rate, economy, run rate, IsHighScoreInnings, team segment
Language and libraries	Python 3, pandas, NumPy, Matplotlib, Seaborn
Tools	Jupyter Notebook, Google Colab, VS Code
Output	Cleaned CSV files, result tables, 11 charts including a dashboard, validation checks, key findings

3.2 Problem Statement

IPL teams and analysts want to know which teams and players perform best, how the game has changed over the seasons and which match factors (toss, batting first or chasing, phase of the innings) influence results. The available data (match results and 260,920 ball-by-ball records from 2008 to 2024) is raw: it contains missing values, franchises recorded under old and new names, inconsistent season and venue labels, super-over balls mixed with normal innings and dismissals that are not credited to the bowler. A Python-based solution is required that imports and cleans this data, builds reliable innings and player statistics, compares teams, players and seasons, presents the results as clear charts and validates the results.

3.3 Objectives

1. To import the raw match and ball-by-ball datasets into Python using pandas and inspect their structure and quality.
2. To identify and correct data quality problems: missing values, inconsistent season, team, city and venue labels, super-over balls and non-bowler dismissals.
3. To merge the two files and build an innings-level table with runs, wickets, boundaries, extras and dot balls.
4. To calculate descriptive statistics, detect outliers with the IQR rule and study the distribution of innings scores.
5. To compare teams by wins, win percentage and total runs, and segment them into Low, Medium and High Value groups.
6. To rank batters by runs and strike rate and bowlers by wickets and economy.
7. To evaluate match factors: the toss, batting first vs chasing, dismissal types, scoring by phase and scoring trends by season.
8. To present the results in charts and a dashboard using Matplotlib and Seaborn and validate them with automated checks.

3.4 Scope of the Project

The project covers data import, inspection, cleaning, merging, feature engineering, statistical analysis, visualisation and validation for all IPL matches from 2008 to 2024. The analysis is descriptive: it uses grouping, rates, percentiles and correlation. Predicting match results with machine learning, live data collection, player auction values and comparison with other T20 leagues are outside the scope of this version and are listed as future enhancements.

4. Proposed System / System Design

4.1 System Workflow

The solution follows a **six-stage data analysis pipeline** (Figure 1). The two raw CSV files are loaded into pandas DataFrames, cleaned, merged and enriched with new features, analysed with grouping and statistics, visualised as charts, and finally validated and summarised. Each stage uses the output of the previous stage, so an error left in an early stage would affect every result after it.

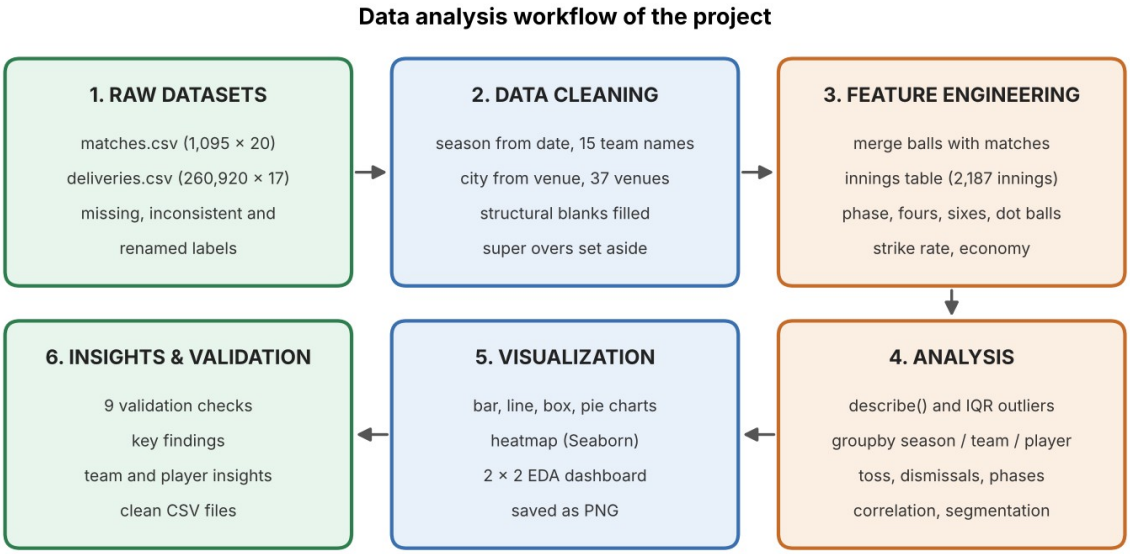


Figure 1: Workflow of the IPL Match & Player Performance Analysis

4.2 Technologies Used

Table 6: Technologies used

Layer	Technology	Purpose in project
Language	Python 3	Complete solution written in Python
Data handling	pandas	read_csv, cleaning, replace, merge, groupby, agg, describe
Numerical	NumPy	Numeric operations and calculations
Visualization	Matplotlib	Chart size, titles, labels, pie chart, saving charts as PNG
Visualization	Seaborn	histplot, boxplot, barplot, heatmap
Statistics	pandas (quantile, skew, corr)	IQR outlier detection, skewness, correlation, terciles

Layer	Technology	Purpose in project
Environment	Jupyter Notebook / Google Colab	Step-by-step execution with visible output
Editor	VS Code	Writing and running the notebook locally

4.3 Dataset Description

The project uses the public **IPL Complete Dataset (2008–2024)** from Kaggle, which consists of two linked files. `matches.csv` contains **1,095 records** (one per match) and **20 columns**; `deliveries.csv` contains **260,920 records** (one per ball) and **17 columns**. The files are linked by the match id (`id` in matches, `match_id` in deliveries); every delivery belongs to a match in the matches file.

Table 7: Dataset columns (grouped)

File	Columns	Description
matches	id, season, date, match_type	Match identifier, season label, date and stage (League, Qualifier, Final …)
matches	city, venue	Where the match was played
matches	team1, team2, toss_winner, toss_decision	The two teams and the toss result (bat or field)
matches	winner, result, result_margin, player_of_match	Winner, won by runs / wickets / tie / no result, margin, best player
matches	target_runs, target_overs, super_over, method, umpire1, umpire2	Chase target, super over flag, D/L method and umpires
deliveries	match_id, inning, batting_team, bowling_team	Match, innings number (3–6 = super over) and teams
deliveries	over, ball, batter, bowler, non_striker	Ball position (over 0–19) and players involved
deliveries	batsman_runs, extra_runs, total_runs, extras_type	Runs from the bat, extras and their type (wides, no-balls, byes …)
deliveries	is_wicket, player_dismissed, dismissal_kind, fielder	Wicket flag and dismissal details

4.4 Data Quality Issues Found

The raw data was inspected with `head()`, `dtypes`, `isnull().sum()`, `duplicated()`, `unique()` and `value_counts()`. The problems found are listed in Table 8.

Table 8: Problems found in the raw dataset

Problem	Column	Example	Count
Missing values	7 columns of matches	city, winner, result_margin, method …	1,160 cells
Structural blanks	4 columns of deliveries	dismissal_kind on a normal ball	994,301 cells
Inconsistent season	season	2007/08, 2009/10, 2020/21	178 matches
Renamed teams	team, toss, winner columns	Delhi Daredevils / Delhi Capitals	19 labels, 15 teams

Problem	Column	Example	Count
City spelling	city	Bangalore / Bengaluru	2 spellings
Duplicate venue names	venue	Wankhede Stadium, Mumbai	58 labels, 37 grounds
Super-over balls	inning 3–6	Tie-breaker overs	161 balls, 14 matches
Non-bowler dismissals	dismissal_kind	run out, retired hurt	1,135 wickets
Duplicate records	All	–	0

4.5 Data Cleaning Steps

1. **Strip spaces** – extra spaces were removed from every text column of both files.
2. **Fix the season** – the date was parsed with `pd.to_datetime()` and the season was taken from the match year, so 2007/08 became 2008, 2009/10 became 2010 and 2020/21 became 2020.
3. **Standardise team names** – four old names were mapped to the current franchise name (Delhi Daredevils → Delhi Capitals, Kings XI Punjab → Punjab Kings, Royal Challengers Bangalore → Royal Challengers Bengaluru, Rising Pune Supergiants → Rising Pune Supergiant) in all six team columns, giving 15 franchises.
4. **Fill and standardise the city** – the 51 blank cities were filled from the venue (Dubai or Sharjah) and Bangalore was changed to Bengaluru.
5. **Standardise venues** – the ", City" suffix was removed and renamed grounds were merged (Feroz Shah Kotla → Arun Jaitley Stadium, Sardar Patel Stadium → Narendra Modi Stadium), reducing 58 venue labels to 37.
6. **Fill structural blanks in matches** – winner → No Result, player_of_match → Unknown, method → Normal, and result_margin, target_runs and target_overs → 0 for ties and no-result matches.
7. **Fill structural blanks in deliveries** – extras_type → None and player_dismissed, dismissal_kind and fielder → Not Out for balls without a wicket.
8. **Set aside super overs** – the 161 balls of innings 3–6 were removed from the innings statistics, leaving 260,759 balls.

Code: data cleaning

```
matches['date'] = pd.to_datetime(matches['date'], errors='coerce')
matches['season'] = matches['date'].dt.year                    # 2007/08 -> 2008
team_map = {'Delhi Daredevils': 'Delhi Capitals', 'Kings XI Punjab': 'Punjab Kings',
            'Royal Challengers Bangalore': 'Royal Challengers Bengaluru',
            'Rising Pune Supergiants': 'Rising Pune Supergiant'}
for col in ['team1', 'team2', 'toss_winner', 'winner']:
    matches[col] = matches[col].replace(team_map)
matches['city'] = matches['city'].fillna(matches['venue'].map(
    {'Dubai International Cricket Stadium': 'Dubai', 'Sharjah Cricket Stadium': 'Sharjah'}))
matches['venue'] = matches['venue'].str.split(',').str[0].str.strip().replace(venue_map)

matches['winner'] = matches['winner'].fillna('No Result')
matches[['result_margin', 'target_runs', 'target_overs']] = matches[[...]].fillna(0)
deliveries['extras_type'] = deliveries['extras_type'].fillna('None')
super_over = deliveries[deliveries['inning'] > 2]             # 161 balls set aside
deliveries = deliveries[deliveries['inning'] <= 2].copy()
```

Table 9 shows how real rows of the dataset changed after each cleaning step.

Table 9: Examples of values before and after cleaning

Cleaning step	Match id	Before	After
Season label	335982	season = 2007/08	2008
Season label	1216492	season = 2020/21	2020
Renamed team	335984	team1 = Delhi Daredevils	Delhi Capitals
Blank city	729281	city = blank (Sharjah Cricket Stadium)	Sharjah
City spelling	335982	city = Bangalore	Bengaluru
Venue suffix	1254059	Wankhede Stadium, Mumbai	Wankhede Stadium
Renamed ground	335984	Feroz Shah Kotla	Arun Jaitley Stadium
No result	501265	winner, player_of_match, margin blank	No Result, Unknown, 0
Tie	392190	result_margin blank (won in super over)	0

4.6 Derived Columns & Metrics

After cleaning, the ball-by-ball data was merged with the match details and grouped into an **innings table** (one row per team per match). New columns were created for the analysis:

- **over_no** = over + 1 (overs 1–20) and **phase** – Powerplay (1–6), Middle (7–15), Death (16–20).
- **is_four, is_six, is_dot** – flags for boundaries and dot balls; summed per innings with runs, wickets and extras.
- **IsHighScoreInnings** = 1 if the innings score is 180 or more.
- **Team segment** – Low, Medium or High Value by total runs, using tercile cut-offs.
- **Strike rate** = runs ÷ balls faced × 100 and **Economy** = runs conceded ÷ legal balls × 6.

Worked example: V Kohli scored **8,004** runs from **6,065** balls faced. Strike rate = 8,004 ÷ 6,065 × 100 = **132.0**, i.e. 132 runs per 100 balls.

Code: merge and innings table

```
df = deliveries.merge(matches[['id', 'season', 'city', 'venue', 'winner', 'toss_winner',
                               'toss_decision']], left_on='match_id', right_on='id',
how='left')
innings = (df.groupby(['match_id', 'season', 'venue', 'batting_team', 'inning'],
as_index=False)
             .agg(innings_runs=('total_runs', 'sum'), wickets=('is_wicket', 'sum'),
                  fours=('is_four', 'sum'), sixes=('is_six', 'sum'),
                  extras=('extra_runs', 'sum'), dot_balls=('is_dot', 'sum')))
innings['IsHighScoreInnings'] = (innings['innings_runs'] >= 180).astype(int)
```

4.7 Team, Player & Match Evaluation

Teams were compared by matches, wins and win percentage, and segmented by total runs. Batters were ranked by runs and bowlers by wickets credited to the bowler. For match factors, the toss

result was compared with the match result, and each ball was placed in one of three phases of the innings (Table 10). The correlation between innings-level columns shows which factors move together with the score.

Table 10: Phases of a T20 innings used in the evaluation

Phase	Overs	Balls analysed
Powerplay	1 – 6	81,744
Middle overs	7 – 15	119,552
Death overs	16 – 20	59,463

Code: analysis and charts

```
bowler_wkts = deliveries[(deliveries['is_wicket'] == 1) &
                         (~deliveries['dismissal_kind'].isin(non_bowler))]   # no run outs
bowl = bowler_wkts.groupby('bowler').size().sort_values(ascending=False).head(10)
tossed['toss_win_match_win'] = tossed['toss_winner'] == tossed['winner']
deliveries['phase'] = pd.cut(deliveries['over_no'], [0, 6, 15, 20],
                             labels=['Powerplay (1-6)', 'Middle (7-15)', 'Death (16-20)'])
corr = innings[['innings_runs', 'fours', 'sixes', 'extras', 'wickets', 'dot_balls']].corr()

sns.histplot(innings['innings_runs'], bins=30, kde=True)          # charts
sns.barplot(x=bat['runs'], y=bat.index)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r')
```

4.8 Implementation Steps

1. Install Python 3 and the libraries: `pip install pandas numpy matplotlib seaborn notebook`.
2. Place `matches.csv` and `deliveries.csv` in the project folder (or upload them in Google Colab).
3. Load the data with `pd.read_csv()` and inspect it with `head()`, `dtypes`, `isnull().sum()` and `duplicated()`.
4. Apply the eight cleaning steps and confirm that no missing values remain.
5. Merge the files and build the innings table with runs, wickets, fours, sixes, extras and dot balls.
6. Explore the data with `describe()`, the IQR rule, a histogram, a box plot and a correlation heatmap.
7. Rank teams, batters and bowlers; evaluate the toss, dismissals and phases; segment the teams.
8. Draw the charts and the dashboard and save all 11 charts in the `charts` folder.
9. Run the validation checks, print the key findings and save `matches_clean.csv` and `innings_summary.csv`.

5. Screenshots of Results

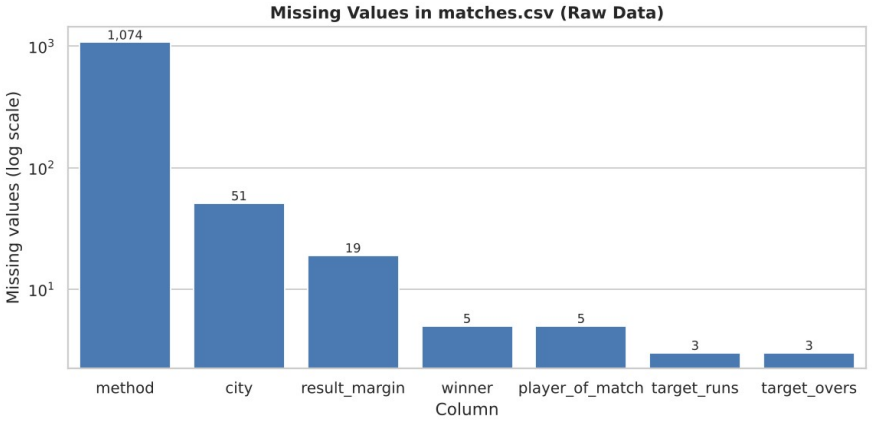
The notebook was run from top to bottom and every cell executed without errors. The main outputs, tables and charts are given below.

5.1 Data Cleaning Result

Table 11 shows some rows of the raw matches file exactly as they were received. Each row has at least one problem, which shows why cleaning was the first step of the project.

Table 11: Sample of the raw matches data with problems (– means blank)

Match id	Season	City	Venue	Team 1	Winner	Problem
335982	2007/08	Bangalore	M Chinnaswamy Stadium	Royal Challengers Bangalore	KKR	Season, city, old team name
335984	2007/08	Delhi	Feroz Shah Kotla	Delhi Daredevils	Delhi Daredevils	Old ground and team name
729281	2014	–	Sharjah Cricket Stadium	Delhi Daredevils	RCB	City missing
501265	2011	Delhi	Feroz Shah Kotla	Delhi Daredevils	–	No result: winner, margin blank
392190	2009	Cape Town	Newlands	KKR	Rajasthan Royals	Tie: margin blank
1254059	2021	Mumbai	Wankhede Stadium, Mumbai	CSK	Delhi Capitals	Venue suffix
1216492	2020/21	Abu Dhabi	Sheikh Zayed Stadium	Mumbai Indians	CSK	Season label



Screenshot 1: Missing values in each column of matches.csv (raw data)

What it shows: the method column is blank in 1,074 matches because only the 21 rain-affected matches use the D/L method. City is missing in 51 matches (all in Dubai and Sharjah), result_margin in 19 (ties and no results), winner and player_of_match in 5 (no results) and the target in 3. In total there are 1,160 missing cells in matches.csv; a log scale is used so the small counts remain visible.

Table 12: Dataset before and after cleaning

Check	Before cleaning	After cleaning
Matches	1,095	1,095
Missing values (matches)	1,160	0
Missing values (deliveries)	994,301	0
Season labels	17 (3 two-year labels)	17 (2008 – 2024)
Team labels	19	15
Venue labels	58	37

Check	Before cleaning	After cleaning
City labels	36	35
Deliveries used	260,920	260,759 (super overs set aside)

Output

```
matches   nulls: 0 | shape (1095, 20)
deliveries nulls: 0 | shape (260759, 17) | super-over balls set aside: 161
teams: 15 | venues: 37 | cities: 35 | seasons: 2008 - 2024
merged rows: (260759, 28) | innings: (2187, 11)
```

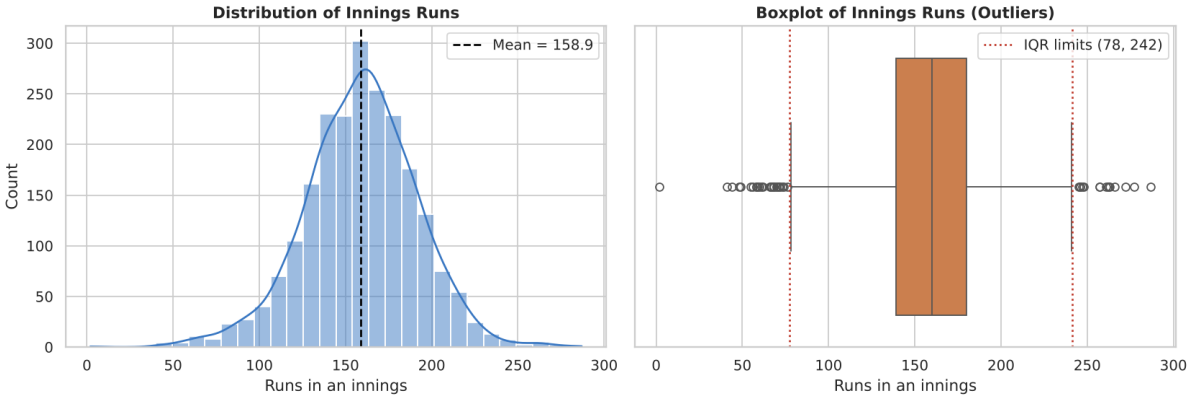
5.2 Innings Statistics

After merging, the data contains **2,187 innings**. Before comparing teams and seasons, the overall shape of the innings scores was studied using `describe()`, a histogram, a box plot and the IQR rule.

Table 13: Descriptive statistics of the innings table (2,187 innings)

Statistic	Runs	Wickets	Fours	Sixes	Extras	Dot balls
Mean	158.88	5.91	13.64	5.96	8.08	41.34
Std. deviation	32.58	2.45	4.21	3.56	4.26	9.07
Minimum	2	0	0	0	0	5
25 % (Q1)	139	4	11	3	5	36
50 % (Median)	160	6	13	6	7	41
75 % (Q3)	180	8	16	8	11	47
Maximum	287	10	31	24	28	74

A typical IPL innings scores about **160 runs** for the loss of **6 wickets**, with 13–14 fours, 6 sixes and 41 dot balls. The mean (158.9) is slightly below the median (160), so a few very low scores pull the average down.

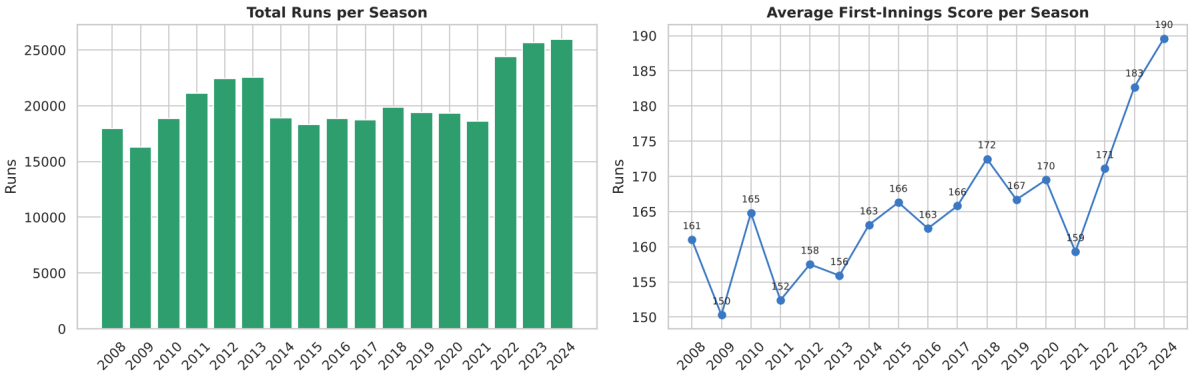


Screenshot 2: Distribution and box plot of innings runs

What it shows: the histogram is close to a bell shape centred around 160 runs, with a slight tail on the left (skewness −0.18). With Q1 = 139, Q3 = 180 and IQR = 41, the box plot limits are 77.5 and 241.5 runs, which flags **43 outlier innings** (28 low and 15 high). The low ones are rain-shortened

or collapsed innings (the 2-run innings is a 2015 match that was washed out), and the high ones are record totals such as **287** by Sunrisers Hyderabad in 2024. They are genuine results, so they were kept.

5.3 Season Trends



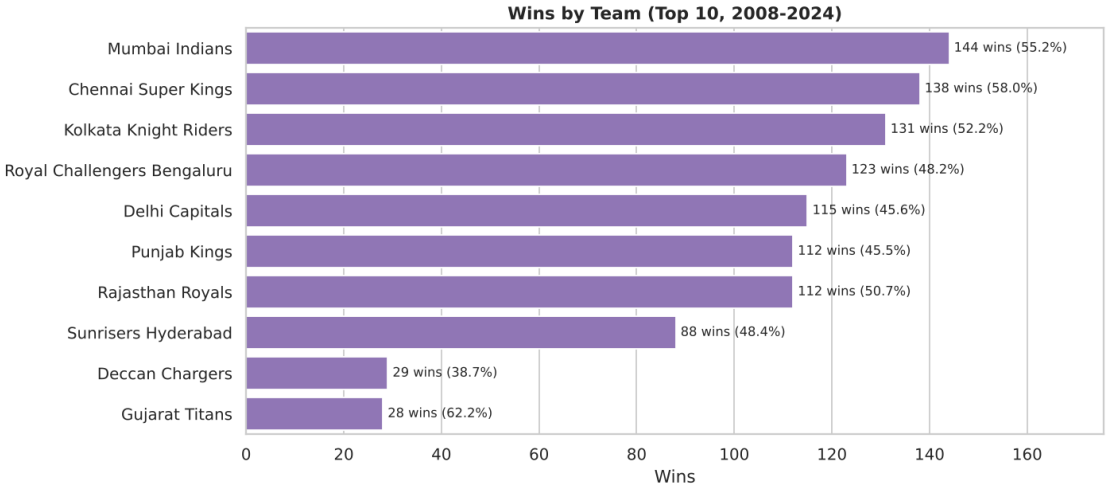
Screenshot 3: Total runs and average first-innings score per season

What it shows: total runs mainly follow the number of matches – seasons with 73–76 matches (2011–2013, 2022 and 2023) have the highest totals. The average first-innings score is a fairer measure: it fell to 150.3 in 2009 (played in South Africa), stayed between about 152 and 173 for most seasons, and then rose sharply to **182.7 in 2023** and **189.6 in 2024**, the highest ever. Sixes per match rose from 10.7 in 2008 to 17.8 in 2024, which shows that T20 batting has become much more aggressive.

5.4 Team Performance

Table 14: Team records 2008–2024 (wins include ties won in a super over)

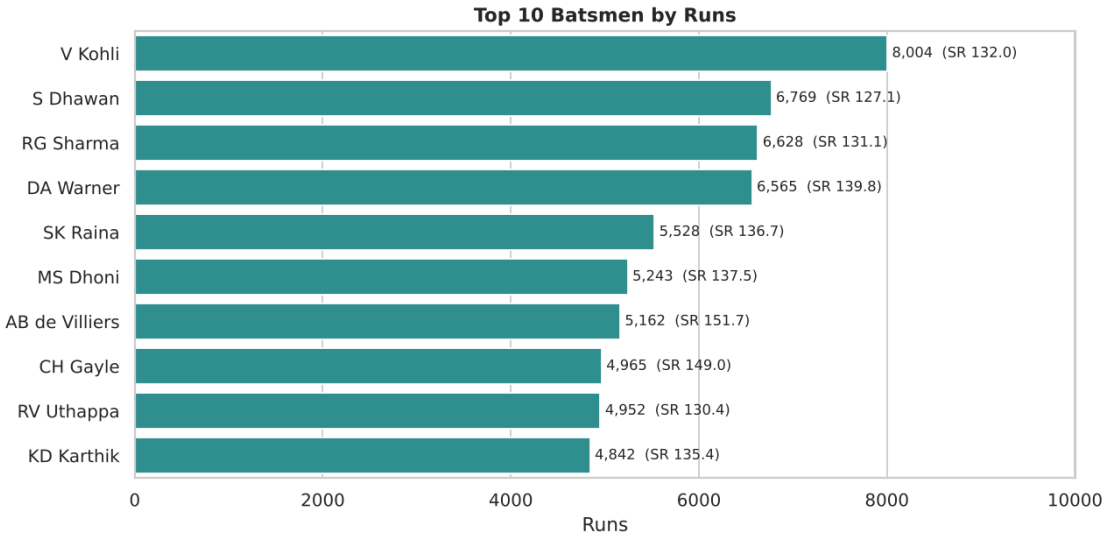
Team	Matches	Wins	Win %	Total runs	Segment
Mumbai Indians	261	144	55.2	42,133	High
Chennai Super Kings	238	138	58.0	38,620	Medium
Kolkata Knight Riders	251	131	52.2	39,295	High
Royal Challengers Bengaluru	255	123	48.2	40,581	High
Delhi Capitals	252	115	45.6	39,164	High
Punjab Kings	246	112	45.5	39,553	High
Rajasthan Royals	221	112	50.7	34,712	Medium
Sunrisers Hyderabad	182	88	48.4	29,034	Medium
Deccan Chargers	75	29	38.7	11,463	Medium
Gujarat Titans	45	28	62.2	7,757	Medium
Lucknow Super Giants	44	24	54.5	7,510	Low
Rising Pune Supergiant	30	15	50.0	4,533	Low
Gujarat Lions	30	13	43.3	4,856	Low
Pune Warriors	46	12	26.1	6,358	Low
Kochi Tuskers Kerala	14	6	42.9	1,901	Low



Screenshot 4: Wins by team (top 10)

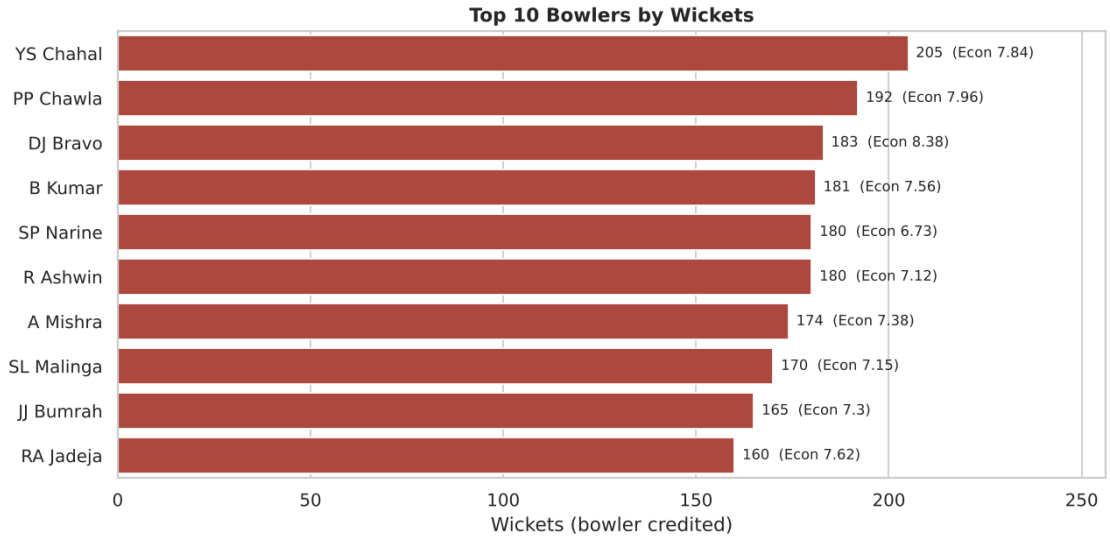
What it shows: Mumbai Indians have the most wins (**144**), followed by Chennai Super Kings (138) and Kolkata Knight Riders (131). Measured by win percentage, however, Chennai (58.0 %) is ahead of Mumbai (55.2 %), and the newest team, Gujarat Titans, has the best record of all (62.2 % in 45 matches). Because total runs mostly depend on how many seasons a team has played, the Low/Medium/High segments (cut-offs 7,675 and 38,801 runs) mainly separate long-standing teams from newer or defunct ones.

5.5 Player Performance



Screenshot 5: Top 10 batsmen by runs

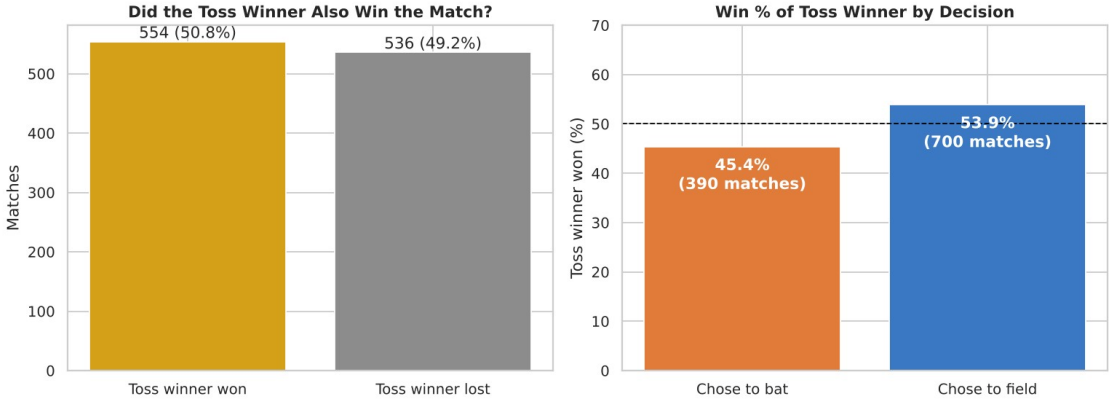
What it shows: V Kohli is the highest run-scorer with **8,004** runs, well ahead of S Dhawan (6,769) and RG Sharma (6,628). Among the top ten, AB de Villiers has the highest strike rate (151.7), followed by CH Gayle (149.0), while Kohli scores at 132.0. AB de Villiers also won the most Player of the Match awards (25).



Screenshot 6: Top 10 bowlers by wickets

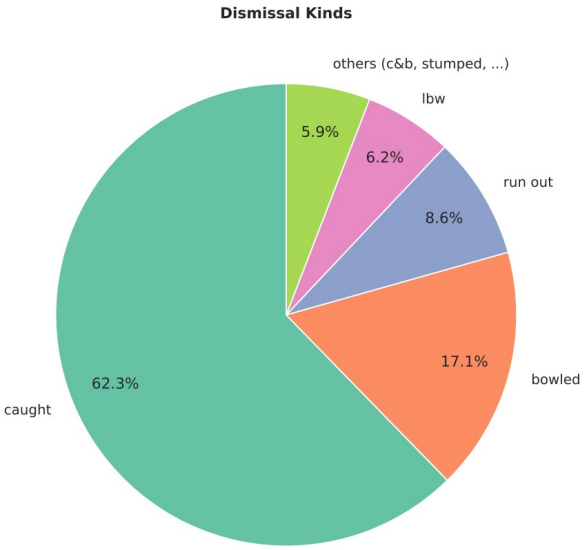
What it shows: YS Chahal leads with **205** wickets, followed by PP Chawla (192) and DJ Bravo (183). Counting every dismissal on a bowler's ball would wrongly give Chahal 213 wickets, because 8 of them were run outs; the cleaned count credits only bowler dismissals. SP Narine has the best economy rate in the top ten (6.73 runs per over).

5.6 Match Factors



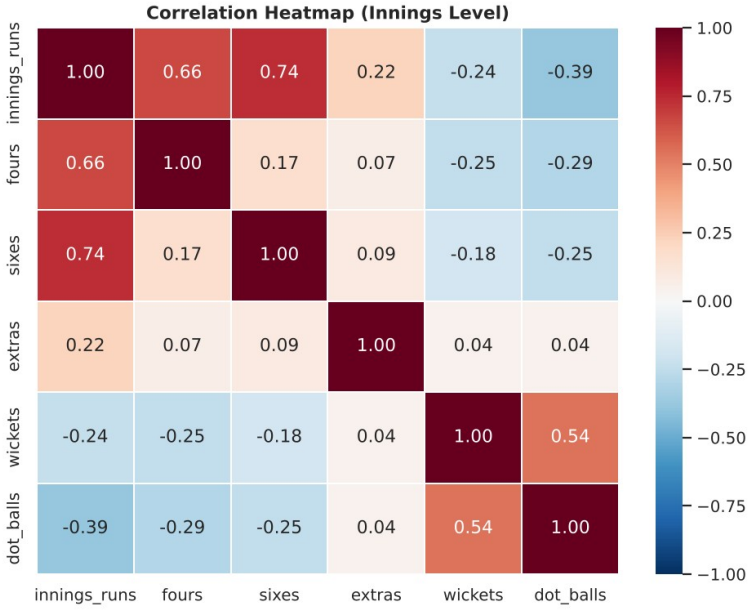
Screenshot 7: Impact of the toss

What it shows: the toss winner won only **50.8 %** of the 1,090 matches with a result – almost the same as tossing a coin, so the toss alone has very little effect. Captains chose to field first in 64.3 % of tosses, and those teams won 53.9 % of the time against 45.4 % for teams that chose to bat. This agrees with the overall result that the chasing team won 53.7 % of decided matches, even though second innings average fewer runs (152.2) than first innings (165.5).



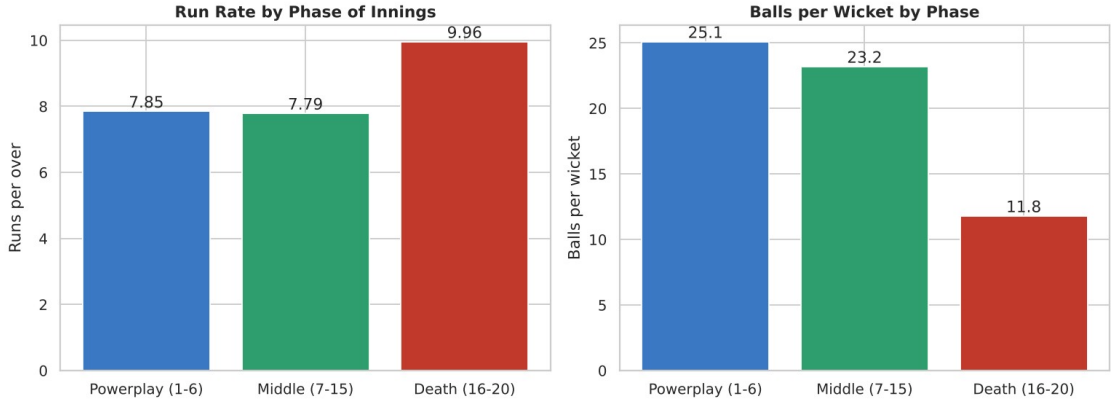
Screenshot 8: Dismissal kinds

What it shows: **caught** is by far the most common dismissal (62.3 %), followed by bowled (17.1 %), run out (8.6 %) and lbw (6.2 %). Caught and bowled and stumped are 2.8 % each. Almost two-thirds of wickets therefore depend on fielders taking catches.



Screenshot 9: Correlation heatmap (innings level)

What it shows: the innings score is most strongly related to **sixes (0.74)** and fours (0.66): boundaries decide the total. Wickets (−0.24) and dot balls (−0.39) are negatively related to the score, and wickets and dot balls rise together (0.54). Extras have only a weak link with the score (0.22).

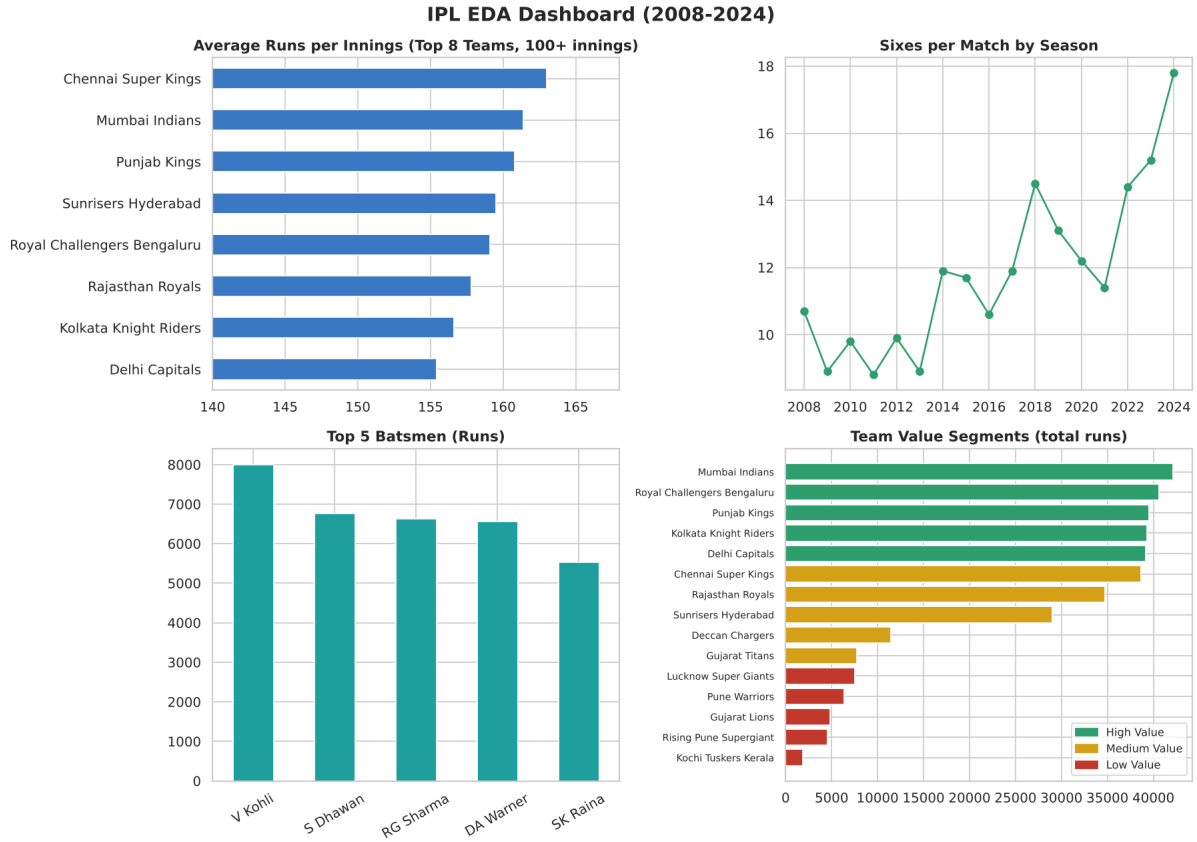


Screenshot 10: Run rate and balls per wicket by phase

What it shows: the run rate is almost the same in the Powerplay (7.85) and the middle overs (7.79), but jumps to **9.96 runs per over** at the death. Wickets also fall much faster at the death – one every 11.8 balls compared with 25.1 in the Powerplay – because batters take more risks in the last five overs.

5.7 EDA Dashboard

The dashboard combines four views on one page so the main patterns can be seen at a glance.



Screenshot 11: IPL EDA dashboard

What it shows: Chennai Super Kings have the highest average innings score among teams with 100 or more innings, sixes per match have risen strongly in recent seasons, Kohli leads the top five run-

scorers, and the team segment chart shows the High Value group made up of five long-standing franchises.

5.8 Validation of Results

The following checks were run in the notebook to confirm that the cleaned data and the calculated results are correct. All checks gave the expected result.

Table 15: Validation tests

Test	Check	Expected result	Result
1	No missing values in matches	matches.isnull().sum().sum() = 0	Pass
2	No missing values in deliveries	deliveries.isnull().sum().sum() = 0	Pass
3	No duplicate matches or balls	duplicated().sum() = 0	Pass
4	Team names standardised	15 unique franchises	Pass
5	Seasons valid	17 seasons between 2008 and 2024	Pass
6	Super overs excluded	Innings table has only innings 1 and 2	Pass
7	Runs add up	Innings runs = ball-by-ball runs (347,470)	Pass
8	Merge complete	Every delivery has a season after the merge	Pass
9	Results add up	Wins + no results = 1,095 matches	Pass

5.9 Key Findings Output

Output

```
================================================================
              IPL ANALYSIS - KEY FINDINGS (2008-2024)
================================================================
1. Matches / innings analysed  : 1,095 / 2,187
2. Total runs (innings 1 & 2)  : 347,470
3. Average innings score       : 158.9 (median 160)
4. Highest innings             : 287 (Sunrisers Hyderabad, 2024)
5. High-score innings (>=180)  : 551 (25.2%)
6. Most wins                   : Mumbai Indians (144 wins, 55.2%)
7. Most runs                   : V Kohli (8,004)
8. Most wickets                : YS Chahal (205)
9. Toss winner won the match   : 50.8%
10. Avg 1st-innings score      : 161.0 in 2008 -> 189.6 in 2024
11. Death-over run rate        : 9.96 vs middle overs 7.79
12. Chasing team won           : 53.7% of 1076 decided (non-tie) matches
13. Avg score 1st vs 2nd inns  : 165.5 vs 152.2
14. Most-used venue            : Wankhede Stadium (118 matches)
15. Most Player of the Match   : AB de Villiers (25 awards)
================================================================
```

6. Conclusion and Future Scope

The project used Python to turn two raw IPL files – 1,095 matches and 260,920 balls from 2008 to 2024 – into clear cricket insights. Data cleaning fixed the season labels, merged 19 team labels into **15 franchises** and 58 venue labels into 37 grounds, filled all 1,160 missing match values with correct values, and kept super overs and non-bowler dismissals out of the statistics. The cleaned

data passed all nine validation checks. The project covers the complete data-science workflow of Python for Data Science: data import, cleaning, merging, feature engineering, grouping, statistics, correlation and visualisation.

Key insights:

1. **Scoring has increased sharply** – the average first-innings score rose from 161.0 in 2008 to a record 189.6 in 2024, and sixes per match from 10.7 to 17.8; 25.2 % of all innings reach 180.
2. **Boundaries decide the total** – the innings score is most strongly correlated with sixes (0.74) and fours (0.66), and negatively with dot balls (−0.39).
3. **The death overs are decisive** – runs come at 9.96 per over in overs 16–20 against about 7.8 earlier, with a wicket every 11.8 balls.
4. **The toss matters little, chasing helps slightly** – the toss winner won 50.8 % of matches, while the chasing team won 53.7 %; most captains (64.3 %) choose to field first.
5. **Consistent teams** – Mumbai Indians have the most wins (144) and Chennai Super Kings the best long-term win rate (58.0 %).
6. **Top performers** – V Kohli (8,004 runs) and YS Chahal (205 wickets) lead the all-time lists; AB de Villiers combines runs with the highest strike rate (151.7) among the top ten.

Future scope:

- Predict match results or first-innings totals with regression or machine learning models.
- Analyse player match-ups (batter vs bowler) and venue-specific scoring patterns.
- Study the effect of the Impact Player rule introduced in 2023 on scoring in more detail.
- Build an interactive dashboard with Plotly or Streamlit for coaches and fans.
- Update the dataset automatically each season from a live cricket data source.

7. References

1. pandas documentation, pandas.pydata.org/docs.
2. NumPy documentation, numpy.org/doc.
3. Matplotlib documentation, matplotlib.org.
4. Seaborn documentation, seaborn.pydata.org.
5. W. McKinney, "Python for Data Analysis", 3rd edition, O'Reilly Media, 2022.
6. Project Jupyter documentation, docs.jupyter.org.
7. Kaggle, "IPL Complete Dataset (2008-2024)" – matches.csv and deliveries.csv, www.kaggle.com.
8. Indian Premier League official website, www.iplt20.com.
9. PBL activity guidelines, Python for Data Science (BE05000231), SVIT Vasad.